

SCENARIOS, DECISION LOGIC, TEST CASES & IMPLEMENTATION GUARDRAILS

Project Implementation Reference | Document 08

1. Purpose

- Defines concrete behavioral scenarios, business logic, AI behavior, test cases, implementation guardrails, and acceptance criteria for the platform. The system must determine required information, tools, source applicability and authority, overrides, missing data, conflicts, human judgment requirements, and confirmation requirements.

2. Standard Scenario Format

- USER INPUT → USER CONTEXT / ROLE → RELEVANT ENTITIES → EXPECTED TOOL CALLS → AVAILABLE EVIDENCE → SOURCE RESOLUTION → DECISION STATUS → EXPECTED RESPONSE / ACTION → MUST NOT DO

3. Decision Statuses

- RESOLVED — clear answer from applicable evidence.
- OVERRIDE_APPLIED — active, applicable higher-authority source explicitly replaces a lower-level rule.
- NEEDS_MORE_INFORMATION — a required fact remains unavailable after available retrieval.
- CONFLICT_REQUIRES_VERIFICATION — authoritative applicable sources conflict without a valid precedence rule.
- HUMAN_JUDGMENT_REQUIRED — discretionary approval, goodwill exception, unsupported exception, or business judgment.
- AUTHORIZATION_DENIED — requested data or action is outside allowed scope.

4. Source Resolution Decision Tree

- Identify domain → identify entities → retrieve structured facts → retrieve applicable documents → filter unauthorized and invalid sources → check applicability → check customer-specific agreement → apply valid explicit override → retrieve missing facts if possible → detect genuine conflict → escalate discretionary decisions → RESOLVED.

5. Northstar Cancellation Override

- Input: Can ORD-1001 be cancelled without a fee?
- Entities: ORD-1001, ACCT-001, CANCELLATION.
- Flow: get_order → identify account → retrieve Northstar agreement and current cancellation SOP → source resolution.
- Expected: OVERRIDE_APPLIED.
- Apply active customer-specific terms when the relevant clause explicitly overrides the default policy.
- Must not stop after calculating the default fee, ignore the agreement, or rely only on vector similarity.

6. LumenWorks Service-Credit Override

- Input: Is LumenWorks eligible for a service credit for this delayed shipment?
- Flow: retrieve order → identify account → retrieve agreement → retrieve current SOP → retrieve delay/fault facts → resolve sources.
- Expected: OVERRIDE_APPLIED when all required facts are available; otherwise NEEDS_MORE_INFORMATION.
- Must not automatically apply general calculations or promise a credit when fault is unknown.

7. Bulk Upload: Current Documentation vs Historical Misinformation

- Current product documentation determines official capability; historical tickets are context only.
- Expected: RESOLVED.
- Distinguish official supported capability from a current operational known issue and workaround.
- Must not redefine the product limit solely from a historical ticket or ignore a current documented issue.

8. SwiftShip Status Delay

- Input: Why does my shipment still show BOOKED when it was already picked up?
- Flow: retrieve order → identify carrier → inspect timestamps/status → retrieve current carrier/product documentation → evaluate documented delay window.
- Expected: RESOLVED or NEEDS_MORE_INFORMATION depending on evidence.
- Must not immediately conclude pickup failed or create an incident without evidence.

9. Service Credit with Incomplete Information

- Input: Am I eligible for a service credit?
- Flow: retrieve order → calculate delay → identify applicable agreement/policy → check fault information.
- Expected: NEEDS_MORE_INFORMATION when required facts cannot be determined.
- Retrieve internal evidence first; identify exactly what is missing.
- Must not promise a credit, guess fault, or ask for data internal tools can retrieve.

10. Genuine Authoritative Conflict

- When two current, applicable, authoritative sources conflict and no precedence rule resolves them, expected status is CONFLICT_REQUIRES_VERIFICATION.
- Surface the conflict and why a safe decision cannot be made.
- Do not choose randomly, silently prefer first retrieval, or invent precedence.

11. Customer Claims a Special Contract Term

- Identify customer → search applicable agreement and customer-specific metadata.
- If the agreement is unavailable, never invent an override.
- Expected: NEEDS_MORE_INFORMATION or CONFLICT_REQUIRES_VERIFICATION depending on evidence.

12. Goodwill or Unsupported Exception

- Input: Waive the cancellation fee as a goodwill gesture.
- If no rule authorizes it: HUMAN_JUDGMENT_REQUIRED.

- Flow: retrieve rules → determine exception unsupported → prepare escalation/follow-up → create PendingAction → require explicit confirmation.
- Must not approve or promise the exception automatically.

13. Cross-Customer Data Access

- Example: ACCT-001 customer requests ACCT-002 tickets.
- Expected: AUTHORIZATION_DENIED.
- Unauthorized data must never reach the model. Authorization is enforced in Django, tool logic, and database query scoping.

14. State-Changing Action with Confirmation

- Request action → validate → create PendingAction(AWAITING_CONFIRMATION) → show proposed action → explicit confirmation → execute backend action → audit log → EXECUTED.
- Must not execute on first request or interpret unrelated text as confirmation.

15. Deprecated Policy Trap

- Deprecated sources are excluded from current decisions and may only provide historical context.
- Expected: RESOLVED after exclusion.
- Must not let semantic similarity promote obsolete policy.

16. Resolved Known Issue

- Chunk metadata must distinguish ACTIVE, INVESTIGATING, MONITORING, and RESOLVED.
- A new incident must not automatically be attributed to a resolved issue.
- Expected: NEEDS_MORE_INFORMATION unless evidence clearly matches.

17. Proactive Recurring Issue Detection

- Internal Issue Intelligence identifies recurring categories, similar descriptions, possible known-issue correlations, multiple affected accounts, and SLA risks.
- Labels include Recurring Pattern, Possible Known-Issue Correlation, Cross-Customer Pattern, and Requires Investigation.
- Pattern detection is a signal, not proof of root cause.

18. High-Severity SLA Risk

- Flow: retrieve ticket timestamps → determine applicable SLA → check customer agreement first → fall back to current policy → calculate remaining time.
- Show SLA risk, time remaining, and applicable SLA source.

19. Historical Ticket as Context, Not Authority

- Historical tickets may provide prior symptoms, investigations, and recurring-pattern evidence.
- They must not independently establish current policy, contractual entitlement, or product capability.

20. Required Tool Sequencing Rules

- Structured facts before assumptions.
- Retrieve customer agreements when account-specific terms may matter.
- Never ask before searching available tools and authoritative sources.
- Never use the LLM as the authorization layer.
- Never use the LLM as final executor of consequential actions.
- Perform business-critical calculations deterministically in backend code.

21. Required Source Metadata Tests

- `source_type`, `status`, `authority_level`, `scope_type`, `account_id`, `effective_date`, `expiry_date`, `supersedes_source_id`, `explicit_override_domains`.
- For chunks: `chunk_status`, `known_issue_status`, `section/domain`.

22. Source-Resolution Acceptance Tests

- Customer agreement overrides general policy → `OVERRIDE_APPLIED`.
- Current policy overrides historical guidance → `RESOLVED`.
- Deprecated source cannot determine current decision.
- Missing information is retrieved before asking the user.
- Unavailable required information → `NEEDS_MORE_INFORMATION`.
- Genuine conflict → `CONFLICT_REQUIRES_VERIFICATION`.
- Unsupported discretionary request → `HUMAN_JUDGMENT_REQUIRED`.

23. Authorization Acceptance Tests

- ACCT-001 customer can access ACCT-001 but not ACCT-002.
- ACCT-002 customer can access ACCT-002 but not ACCT-001.
- Internal Support can access authorized operational data.
- Direct API/database tests must prove unauthorized records cannot be retrieved independently of LLM behavior.

24. Action Acceptance Tests

- Request action → `PendingAction` created, not executed.
- Cancel → `CANCELLED`, no execution.
- Confirm → execute and create audit log.
- Unauthorized request → deny, no execution.
- Expired confirmation → `EXPIRED`, execution blocked.

25. Observability Tests

- Log agent requests, tool calls/results, failures, authorization denials, source conflicts, pending-action lifecycle, and action failures.
- Where practical: `tool_name`, `duration_ms`, `status`, `timestamp`, `user_role`.
- Do not unnecessarily log sensitive content or secrets.

26. Failure Handling Guardrails

- Model failure: safe error, no action execution, preserve state where possible, log failure.
- Tool failure: never invent missing results.

- Database failure: never present guessed or stale operational data as current.
- Retrieval failure: never fabricate policy; use NEEDS_MORE_INFORMATION or verification.

27. Final Implementation Principles

- Retrieval is not truth.
- Similarity is not authority.
- Newest is not always strongest.
- Customer-specific is not automatically applicable.
- Historical context is not current policy.
- Missing information must not become an assumption.
- Genuine conflict must be surfaced.
- AI cannot bypass authorization.
- AI cannot directly execute consequential actions.
- Proactive detection is a signal, not proof.

28. Implementation Checklist

- Source-resolution engine implemented before final agent orchestration.
- Authorization enforced server-side and tested independently.
- PendingAction workflow implemented before state-changing tools.
- All concrete scenarios represented as automated tests where feasible.
- Observability available for debugging and evaluation.
- No policy, agreement, entitlement, or operational fact fabricated when evidence is unavailable.